Open-Book Quiz

Distributed Leak-Risk Propagation with MPI and Checkpoint/Restart

Parallel and Distributed Computing (CS-3006)

Online Quiz • Open Book • Individual Submission

Time	As announced by the instructor
Marks	20
Resources	Open book, notes, course slides, and your own practice material
Submission	One .c file (or a small set of .c/.h files), plus a short design note

Instructions

- Read the problem statement carefully. The update rule and output requirements are custom.
- Write a distributed-memory MPI solution.
- Your solution must use both point-to-point communication and collective communication where appropriate.
- You must also add simple checkpoint/restart-based fault tolerance.
- Assume restart happens with the same number of MPI processes as the original run.

Case Study: Distributed Leak-Risk Propagation on a Pipe Grid

A rectangular grid models a monitored pipe network. Each cell stores a leak-risk score. It is just a **grid-based simulation** where some places are “danger spots,” and that danger slowly spreads to nearby places over time.

Imagine a board of squares:

- some squares are **safe**
- some squares are **blocked**
- some squares are **leak source points**
- every other square has a **risk value**

At every step:

- each square looks at its nearby squares
- it updates its own risk based on neighboring risk
- so the “leak risk” gradually spreads across the board

The grid has M rows and N columns. Each cell is one of the following:

- -1 = blocked / broken wall cell (never changes)
- 100 = leak source cell (fixed forever)
- Any other nonnegative value = current leak-risk score

At each iteration, every non-blocked, non-source interior cell is updated using:

$$new[i][j] = 0.40 \cdot old[i-1][j] + 0.20 \cdot old[i+1][j] + 0.15 \cdot old[i][j-1] + 0.15 \cdot old[i][j+1] + 0.10 \cdot old[i][j]$$

Additional custom rule:

- If a cell has two or more blocked neighbours, then after computing the above value, multiply it by 0.8.

Alarm rule:

- After each iteration, any non-blocked cell with value >= THRESHOLD is considered an alarm cell.

The simulation stops when either of the following occurs:

- the global maximum change is smaller than eps (**stopping tolerance,** a very small value used to decide:"Have the grid values almost stopped changing?")
- max_iters is reached

At rank 0, your program must finally print:

- total number of iterations executed
- global maximum final cell value
- total number of alarm cells
- checksum of the final grid (ignoring blocked cells)

Given Serial Baseline

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>

static double max2(double a, double b) { return (a > b) ? a : b; }

void init_grid(double *A, int M, int N) {
    for (int i = 0; i < M; i++) {
        for (int j = 0; j < N; j++) {
            if (i == 0 || i == M-1 || j == 0 || j == N-1)
                A[i*N + j] = 100.0;   // fixed boundary
            else
                A[i*N + j] = 0.0;
        }
    }
    if (M > 6 && N > 6) {
        A[3*N + 3] = -1.0;
        A[3*N + 4] = -1.0;
        A[4*N + 3] = -1.0;
    }
}

int blocked_neighbors(const double *A, int M, int N, int i, int j) {
    int c = 0;
    if (A[(i-1)*N + j] < 0) c++;
    if (A[(i+1)*N + j] < 0) c++;
    if (A[i*N + (j-1)] < 0) c++;
    if (A[i*N + (j+1)] < 0) c++;
    return c;
}
```

```
int main(int argc, char **argv) {
    int M = (argc > 1) ? atoi(argv[1]) : 12;
    int N = (argc > 2) ? atoi(argv[2]) : 12;
    int max_iters = (argc > 3) ? atoi(argv[3]) : 1000;
    double eps = (argc > 4) ? atof(argv[4]) : 1e-3;
    double THRESHOLD = (argc > 5) ? atof(argv[5]) : 60.0;

    double *A = (double*)malloc((size_t)M * N * sizeof(double));
    double *B = (double*)malloc((size_t)M * N * sizeof(double));
    if (!A || !B) {
        fprintf(stderr, "Allocation failed\n");
        return 1;
    }

    init_grid(A, M, N);
    init_grid(B, M, N);

    int iter;
    for (iter = 0; iter < max_iters; iter++) {
        double diff = 0.0;
        for (int i = 1; i < M-1; i++) {
            for (int j = 1; j < N-1; j++) {
                if (A[i*N + j] < 0 || A[i*N + j] == 100.0) {
                    B[i*N + j] = A[i*N + j];
                    continue;
                }
                B[i*N + j] =
                    0.40 * A[(i-1)*N + j] +
                    0.20 * A[(i+1)*N + j] +
                    0.15 * A[i*N + (j-1)] +
                    0.15 * A[i*N + (j+1)] +
                    0.10 * A[i*N + j];
                if (blocked_neighbors(A, M, N, i, j) >= 2)
                    B[i*N + j] *= 0.8;
                diff = max2(diff, fabs(B[i*N + j] - A[i*N + j]));
            }
        }
        for (int i = 1; i < M-1; i++) {
            for (int j = 1; j < N-1; j++) {
                if (A[i*N + j] >= 0 && A[i*N + j] != 100.0)
                    A[i*N + j] = B[i*N + j];
            }
        }
        if (diff < eps) break;
    }

    double checksum = 0.0, maxv = -1.0;
    int alarms = 0;
    for (int i = 0; i < M*N; i++) {
        if (A[i] >= 0) {
            checksum += A[i];
            if (A[i] > maxv) maxv = A[i];
            if (A[i] >= THRESHOLD) alarms++;
        }
    }
    printf("iters=%d max=%.6f alarms=%d checksum=%.6f\n", iter, maxv, alarms, checksum);

    free(A);
    free(B);
    return 0;
}
```

Tasks

Part A: Distributed MPI version

- Use decomposition strategy appropriate for the problem.
- Rank 0 initializes the full grid.
- Use point-to-point communication for top/bottom halo exchange.
- Use collective communication for global convergence and/or final statistics.
- The final result at rank 0 must match the serial semantics.

Part B: Checkpoint/Restart fault tolerance

- Every CKPT_INTERVAL iterations, save a checkpoint.
- Each rank must save at least: current iteration number and its local grid rows.
- On restart, if checkpoint files exist, resume from the latest saved iteration instead of starting fresh.
- Checkpointing must be coordinated at consistent iteration boundaries.
- Use files named like ckpt_rank_0.bin, ckpt_rank_1.bin, and so on.

Expected submission

- One .c file (or a small set of .c/.h files) implementing the MPI solution
- A short design note explaining decomposition, halo exchange, collectives used, and checkpoint format
- Compile command and one sample run command

Marks distribution

Component	Marks
Appropriate & Correct decomposition	3
Correct communicate/exchange	4
Correct update rule including blocked/source logic	4
Global convergence + final statistics using collectives	3
Checkpoint/restart correctness	4
Design-note clarity	2